Chrome

Double Hi-Res Graphics Commands For Applesoft

Zachary T. Smith

If you are frustrated by the lack of firmware support for the latest Apple graphics modes, you'll be more than pleased with this Applesoft BASIC extension that gives you six new graphics commands for the double hi-res mode on the Apple IIe (Revision B), IIc, and IIGS computers. DOS 3.3 is required.

Here's a utility that extends Applesoft with six new commands and a revised HPLOT to provide full control over the double hi-res graphics mode. It supports both monochrome and 16-color plotting and leaves 21.5K of RAM for the Applesoft programmer.

The whole idea of double high-resolution video (including 80-column text) on the Apple seems to be more of an afterthought than an innovation. For example, the original ROM routine for scrolling the 80-column screen behaved more like two successive 40-column scrolls on the same area of RAM (with first the main RAM and then the auxiliary RAM switched in). While some ROM routines have been revised to improve support for double high-resolution graphics, Applesoft BASIC has not been upgraded. As a result, there is no easy way for BASIC programmers to take advantage of the double hi-res mode. "Chrome" changes all that.

Chrome is a utility which extends Applesoft. It adds several new commands and revises some old ones to allow full exploitation of the double hi-res graphics mode of the 128K IIe, IIc, and IIGS. It can be used with either color or monochrome monitors to support screen displays of 140 × 192 pixels in 16 colors or 560 × 192 pixels in monochrome.

Chrome does not support the new medium-resolution graphics mode (80 × 48 pixels in 16 colors). However, that mode can be emulated in double hi-res.

Creating The Startup Disk

The simplest way to use Chrome is to dedicate a disk to it. Format a fresh disk under DOS 3.3. Next, type in the following short program:

```
10 PRINT CHR$ (4)"PR#3"
15 PRINT "DOUBLE HI-RES EXTENDE
   D APPLESOFT BEING LOADED."
20 PRINT CHR$ (4)"BRUN CHROME"
30 NEW
```

To have this setup program run automatically when the disk is booted, save the program to the disk with the name HELLO.

Since Chrome is written entirely in machine language, it should be entered with the Apple version of the "MLX" machine language entry program, found elsewhere in this issue. Be sure you read and understand the instructions for using MLX before you begin entering data. When you run MLX, you'll be asked for a starting address and an ending address. For Chrome (Program 1), give the following values:

```
STARTING ADDRESS?    1800
ENDING ADDRESS?      1FA7
```

When you finish typing in all the data from Program 1, be sure to save a copy before leaving MLX. When saving, use the filename CHROME, since that is the name the setup program above looks for. Be sure to save the final copy of the program on the same disk as the setup program.

To use Chrome, turn off your computer, place in the drive the disk containing the setup HELLO program and the CHROME file, and turn on the system. With this disk booted, you'll be provided with a new version of Applesoft with three distinguishing characteristics: First, only 21.5K of RAM is available for your programs. Second, the 80-column text screen is now the default display. And third, several new commands and enhancements are available. Only a Control-Reset will disengage the new BASIC and return you to standard Applesoft.

If at any time something goes wrong with BASIC, the recover procedure is to press Control-Reset as usual, followed by CALL 6144 to reinitialize Chrome's BASIC extension and its new tokens. Neither the reset nor the call performs a NEW, so the residing BASIC program is still there afterwards.

If you feel adventuresome, skip ahead to the description of the commands and start trying some of them. If you would rather start with a demo program, type in, save, and run Program 2. If you have a color

monitor, switch it to monochrome if such a switch is available. The first part of the demo uses monochrome graphics. It draws ten circles, a line, and a string of box-shaped characters. Switch your monitor back to color and press Return. You'll see a demonstration of the color mode. It draws 16 lines in different colors and then a blue oval. Press Return again to exit the demo.

Using the New Commands

If you've examined Program 2, you have already seen some of the new commands. Let's take a close look at each of them.

HGR 3 sets up a double hi-res screen just as HGR sets up normal hi-res, including the four-line text window at the bottom of the screen. To switch back to 40 columns from within a running program you must give DOS a PR#0 command, such as:

```
PRINT CHR$(4) "PR#0" : HOME : HGR2
```

If DOS is not informed of the change, the system will do strange things. Actually, this makes perfect sense—when you type PR# from the direct mode, it is not only executed as an Applesoft command but also as a DOS 3.3 command.

HGR 4 is the same as HGR3, except that it sets up a full graphics screen with no text window at the bottom. Unlike HGR2 (its normal hi-res equivalent), HGR 4 uses memory from hi-res page 1.

DOUBLE puts the HPLOT command in 560-point monochrome mode. In this mode, each plot affects only one bit of screen memory. The particular effect of each plot is determined by the most recent HCOLOR= statement (see the discussion below of HCOLOR=). This mode is most useful in conjunction with a monochrome monitor, where each bit of screen memory corresponds to one screen pixel. On a color monitor, four bits are required to show one colored dot. In DOUBLE mode with a color monitor, changing one bit changes a screen dot's color.

COLOR puts the HPLOT command in 140-point color mode. In this mode, each plot affects four bits of screen memory. The value specified in the most recent COLOR= statement (see below) will be stored in the four bits of screen memory at the position specified by the HPLOT command. This mode is most useful in conjunction with a color monitor, where each four-bit group in screen memory specifies the color of one screen pixel. (Since 16 different colors can be displayed, four bits are required to select the color. Four bits can hold values in the range 0–15.) With a monochrome monitor, each bit of screen memory corresponds to one screen pixel, so in COLOR mode with a monochrome monitor each plot affects four screen dots.

HCOLOR= specifies the way that the HPLOT and ELLIPSE commands affect screen memory bits (and hence screen pixels) in the DOUBLE (560-point monochrome) mode. The syntax for the statement is

HCOLOR=*n*

where *n* can have one of the following values:

Value	Effect of plot
0 or 4	Set bit
1 or 5	Clear bit
2 or 6	Invert (toggle) bit
3 or 7	Read bit and store its value (0 or 1) in location 6

For a monochrome screen, setting a bit will make the corresponding pixel appear white, while clearing a bit will make the corresponding pixel black. After a bit is read, a PEEK(6) can be used to determine the current state of the corresponding pixel. HCOLOR=0 is the default setting after an HGR 3 or HGR 4 statement.

COLOR= selects the color to be used when HPLOT or ELLIPSE draws points in COLOR mode. The syntax for the statement is

COLOR=*n*

where *n* is a value in the range 0–15. The correspondence between numbers and colors is as follows:

0	black	8	dark blue
1	magenta	9	purple
2	brown	10	gray 2
3	orange	11	pink
4	dark green	12	medium blue
5	gray 1	13	light blue
6	green	14	aqua
7	yellow	15	white

Chrome provides no facility to directly read the color of a pixel in COLOR mode. However, color values can be determined by switching to DOUBLE mode, specifying HCOLOR=3, and performing four successive reads with HPLOT to collect the four bits of the color value.

HPLOT uses the same syntax as the standard Applesoft version:

HPLOT *x1,y1* [TO *x2,y2* [TO *x3,y3* ...]]

However, the range of allowable values for the horizontal *(x)* coordinates is extended to 0–559.

ELLIPSE can be used to draw ellipses, circles, and similar shapes. It can also be used to draw arcs and other curved lines. The syntax for the statement is

ELLIPSE *x radius, y radius, quadrant* [AT *x center, y center*]

The AT part is optional. The *x center* and *y center* values specify the center of the figure. If omitted, the ellipse will be centered at the current pixel cursor position. This allows faster recursive ellipse drawing around a central point.

The *x radius* and *y radius* values are not checked for validity, but an ellipse can go off the screen without problems if the values are not unreasonably large.

Some other drawing utilities allow you to specify the starting angle and ending angle when drawing portions of ellipses or circles. For simplicity, ELLIPSE only allows you to choose whether or not to draw each of the four quadrants of the figure.

You specify which quadrants to draw using the *quadrant* value in the ELLIPSE statement. The following table shows which *quadrant* values draw corresponding quadrants of the figure:

upper left	8
upper right	4
lower left	2
lower right	1

The values are cumulative. For example, a *quadrant* value of 15 (8 + 4 + 2 + 1) is required to draw a complete ellipse. A value of 9 (8 + 1) would result in only the upper left and lower right quadrants being drawn.

Bitmapped Images

Chrome also provides a way to place bitmapped images onto the double hi-res screen. This method of shape generation is much different than the shape capabilities built into the computer. Bitmapped shape tables are easier to design and can be drawn more rapidly by the system. Shape rotations are

performed by switching between bitmaps rather than by extra computation when the shape is drawn. You may have to experiment a while before you feel comfortable with the next commands. Beginners may wish to ignore the SHAPE command.

The new command has the following syntax:

SHAPE *width, height, location* [AT *x, y*]

The AT portion of the statement is optional. If it is included, the *x* and *y* values specify the coordinates at which the upper left corner of the shape will be drawn.

The *width* value is added to the *x* coordinate after each SHAPE execution. You can use this value to specify the horizontal spacing of shapes on the screen.

The *height* value specifies the height of the bitmapped image in screen lines. All bitmaps are two bytes (14 screen pixels) wide, so the height is half the length of the bitmapped shape table.

The *location* value specifies the starting memory location for the bitmap shape table in RAM. More information on Chrome's memory usage is provided later in this article.

If the *x* coordinate for the shape does not begin on an exact byte boundary (pixel position 0, 7, 14, 21, and so on), the data from the bitmap must be preshifted before drawing. Preshifting does not retain the shifted bits. For instance if a full 14-bit-wide shape is to be plotted at coordinates 55,10, then six pixels will be lost during shifting because the preshifter must move the shape over 6 bits (55 MOD 7 = 6). Only shapes less than 8 bits wide can be drawn anywhere horizontally without risk of being partly lost. All bitmap data is in the same format as screen RAM, with bit 7 unused.

The drawing routine simply takes the data from a buffer and places it on the screen in exclusive-OR mode. Shapes are drawn the same whether DOUBLE or COLOR plotting is enabled.

Memory Usage

Machine language programmers may appreciate the new commands as much as BASIC programmers will. Following is a description of how memory is allotted when Chrome is being used.

The ellipse routine keeps most of its data in the lower 16 bytes of the auxiliary zero page. Main-memory zero-page use is limited to those locations also used by the standard hi-res routines.

The program itself begins at location $1800 hex (6144 decimal) and ends at $1FFF. Screen RAM is at $2000–$3FFF.

When the program is BRUN (loaded and run) it initializes itself in five steps. First, it copies the high ROM ($D000 to $FFFF), including Applesoft, into the main memory RAM and alters Applesoft to accommodate the new commands.

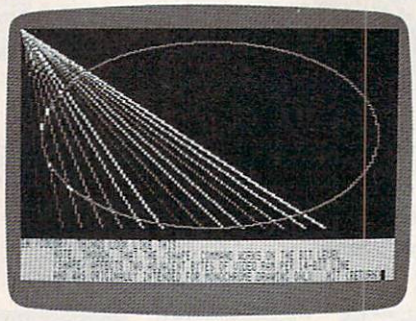
Once the copy-and-change phase is done, four tables are created. Two reside in main-memory high RAM and are switched in when needed. They hold video addresses to speed up the pixel plotter. The third table, located at $1600, is the new tokenizer table; it holds the ASCII keywords of all of the old commands as well as those of the new commands. The fourth table, located at $1400, is used by the ELLIPSE routine only and holds the 16-bit squares of 0–255. These values are also used to speed up pixel position calculations.

The area from $0800 to $13FF is free for machine language routines and bitmaps of character sets.

Program 1: Chrome

For instructions on entering this program, please refer to the "Apple MLX" article elsewhere in this issue.

```
1800: A0 00 84 60 A9 D0 85 61 14
1808: 8D 89 C0 2C 89 C0 2C 89 6D
1810: C0 B1 60 91 60 C8 D0 F9 F3
1818: E6 61 D0 F5 A0 00 B9 2A 30
1820: 19 99 8F 17 F0 03 C8 D0 9C
1828: F5 A0 00 B9 D0 D0 99 00 14
1830: 16 C8 D0 F7 A0 8F B9 CF BD
1838: D1 99 FF 16 88 D0 F7 A9 3A
1840: 00 8D 93 D5 8D 3D D7 A9 5E
1848: 15 8D 97 D5 8D 41 D7 A9 81
1850: 4C 8D 2E D8 A9 42 8D 2F FD
1858: D8 A9 19 8D 30 D8 A9 5C EF
1860: 8D 26 D0 A9 19 8D 27 D0 B3
1868: A9 67 8D 22 D0 A9 19 8D 08
1870: 23 D0 A9 7E 8D 20 D0 A9 BB
1878: 19 8D 21 D0 A0 00 84 67 3F
1880: AD 97 19 18 69 20 85 68 51
1888: 98 91 67 E6 67 8D 80 C0 F7
1890: AD 81 C0 AD 81 C0 A9 00 4D
1898: 85 E6 AA A8 8D B0 18 8D 12
18A0: B2 18 8D B6 18 A9 D0 8D E3
18A8: B3 18 A9 D4 8D B7 18 A9 60
18B0: 6D 99 00 D3 8A 99 00 D7 CD
18B8: E8 E0 07 D0 05 EE B0 18 E0
18C0: A2 00 C8 D0 EA EE B3 18 FA
18C8: EE B7 18 AD B3 18 C9 D3 A1
18D0: D0 DD A0 00 98 48 A2 00 20
18D8: A0 00 20 11 F4 68 A8 A5 AE
18E0: 26 99 00 D8 A5 27 99 00 15
18E8: D9 C8 C0 C0 90 E6 A9 08 D7
18F0: 8D 7F 1D A9 00 8D 09 C0 0F
18F8: 48 85 00 85 03 A9 00 85 4B
1900: 01 85 04 A2 06 A0 03 20 97
1908: 62 1D 68 AA A5 06 A4 07 FF
1910: 8D 08 C0 9D 00 15 98 9D 20
1918: 00 14 E8 8A D0 D7 8D 88 9E
1920: C0 60 C7 19 CD 19 0F 1D 63
1928: 3E 1F 43 4F 4C 4F D2 44 28
1930: 4F 55 42 4C C5 53 48 41 B9
1938: 50 C5 45 4C 4C 49 50 53 EC
1940: C5 00 C9 40 B0 03 4C 32 EE
1948: D8 E9 6B B0 03 4C C9 DE 95
1950: 0A A8 B9 23 19 48 B9 22 9A
1958: 19 48 4C B1 00 2C 92 19 BC
1960: 30 03 4C FE F6 4C D4 19 90
1968: C9 34 D0 03 4C 04 1A C9 47
1970: 33 D0 03 4C 0A 1A 4E 92 7D
1978: 19 20 88 19 4C E2 F3 4E 06
1980: 92 19 20 88 19 4C D8 F3 6E
1988: AD 54 C0 AD 5F C0 8D 0C BE
1990: C0 60 80 00 00 00 00 20 6B
1998: 00 00 01 02 04 08 10 20 8B
19A0: 40 20 67 DD 20 52 E7 A5 85
19A8: 51 A6 50 C9 02 90 06 D0 03
19B0: 50 E0 30 B0 4C 48 8A 48 35
19B8: A9 2C 20 C0 DE 20 F8 E6 2B
19C0: E0 C0 B0 3D 68 A8 68 60 94
19C8: A9 80 8D 98 19 60 A9 00 C8
19D0: 8D 98 19 60 C9 C1 F0 13 63
19D8: 20 A1 19 86 E2 84 E0 85 7F
19E0: E1 20 4E 1A 20 B7 00 C9 21
19E8: C1 D0 12 20 C0 DE 20 A1 D7
19F0: 19 86 E3 84 E6 85 E7 20 53
19F8: F3 1A 4C E4 19 8D 54 C0 EB
1A00: 60 4C 06 F2 AD 52 C0 4C EB
1A08: 0D 1A AD 53 C0 AD 50 C0 52
1A10: AD 57 C0 AD 5E C0 8D 01 F5
1A18: C0 8D 0D C0 A9 00 85 E0 F6
1A20: 85 E1 85 E2 85 E4 A8 AE 2E
1A28: 97 19 86 27 84 26 A2 1F D2
1A30: 8D 54 C0 91 26 C8 D0 FB 63
1A38: 8D 55 C0 91 26 C8 D0 FB AB
1A40: E6 27 CA 10 EB 8E 92 19 E3
1A48: 8D 54 C0 4C B1 00 2C 98 B3
1A50: 19 10 19 A5 E0 48 A5 E1 E7
1A58: 48 46 E1 66 E0 46 E1 66 2F
1A60: E0 20 00 1C 68 85 E1 68 54
1A68: 85 E0 60 60 A5 E4 4A 29 28
1A70: 03 8D 99 19 2C 98 19 10 54
1A78: 03 4C 00 1C A6 E2 E0 C0 46
1A80: F0 E9 A5 E1 C9 02 90 0A FB
1A88: F0 02 B0 DF A5 E0 C9 30 3E
1A90: B0 D9 8D 80 C0 BD 00 D8 23
1A98: 85 26 BD 00 D9 0D 97 19 1C
1AA0: 85 27 A4 E0 A5 E1 09 D0 9B
1AA8: 8D B2 1A 09 D4 8D B5 1A 86
1AB0: B9 00 D0 BE 00 D4 8D 88 BE
1AB8: C0 2C 93 19 10 01 60 4A EB
1AC0: A0 01 90 01 88 99 54 C0 BB
1AC8: A8 BD 9A 19 AE 99 19 F0 A4
1AD0: 10 E0 02 90 11 F0 16 A2 A9
1AD8: 01 31 26 D0 01 CA 86 06 F1
1AE0: 60 11 26 91 26 60 49 7F 2C
1AE8: 31 26 91 26 60 51 26 91 F9
1AF0: 26 60 00 2C 98 19 10 16 72
1AF8: 46 E1 66 E0 46 E1 66 E0 0B
1B00: 46 E7 66 E6 46 E7 66 E6 14
1B08: A9 1B 48 A9 F6 48 A2 FF 9B
1B10: 8E F2 1A E8 A0 02 A5 E6 5B
1B18: 38 E5 E0 85 D0 A5 E7 E5 2B
1B20: E1 B0 12 49 FF AA A5 D0 11
1B28: 49 FF 69 01 90 01 E8 85 20
1B30: D0 8A A2 01 38 85 D1 A5 F6
1B38: E3 E5 E2 B0 06 49 FF 69 FF
1B40: 01 A0 03 85 D2 A5 D0 C5 6C
1B48: D2 A5 D1 E9 00 B0 14 86 9B
1B50: D3 84 1A A6 1A A5 D0 A4 EC
1B58: D2 85 D2 84 D0 A9 00 85 AE
1B60: D1 A4 D3 A5 D1 85 1C 4A A4
1B68: 85 D5 A5 D0 85 1B 6A 85 8B
1B70: D4 BD EF 1B 8D 96 1B B9 E6
1B78: F3 1B 8D D4 1B 2C F2 1A F7
1B80: 30 03 20 6C 1A 4E F2 1A 64
1B88: A5 1B 38 E9 01 85 1B A5 F7
```



Add a wealth of double hi-res graphics commands to Applesoft BASIC with this superb utility.

```
1B90: 1C E9 00 85 1C B0 FE 60 A9
1B98: E6 E0 D0 14 E6 E1 B0 10 06
1BA0: A5 E0 D0 02 C6 E1 C6 E0 48
1BA8: B0 06 C6 E2 B0 02 E6 E2 FD
1BB0: A5 D4 18 65 D2 85 D4 90 2F
1BB8: 02 E6 D5 A5 D0 C5 D4 A5 AB
1BC0: D1 E5 D5 B0 B8 A5 D4 38 5D
1BC8: E5 D0 85 D4 A5 D5 E5 D1 46
1BD0: 85 D5 38 B0 FE E6 E0 D0 77
1BD8: A4 E6 E1 B0 A0 A5 E0 D0 90
1BE0: 02 C6 E1 C6 E0 B0 96 C6 30
1BE8: E2 B0 92 E6 E2 B0 8E 01 75
1BF0: 09 17 13 00 08 16 12 06 96
1BF8: E0 26 E1 06 E0 26 E1 60 89
1C00: A5 E0 48 A5 E1 48 06 E0 C3
1C08: 26 E1 06 E0 26 E1 A5 30 CE
1C10: 85 E8 A9 04 85 E9 06 E8 83
1C18: A2 01 90 01 CA 8E 99 19 E0
1C20: 20 7C 1A E6 E0 D0 02 E6 6E
1C28: E1 C6 E9 D0 E9 68 85 E1 2B
1C30: 68 85 E0 60 00 00 00 00 20
1C38: 00 00 00 00 00 00 00 00 70
1C40: 00 00 00 00 00 00 00 00 78
1C48: 00 00 00 00 00 00 00 00 80
1C50: 00 00 00 00 00 00 00 00 88
1C58: 00 00 00 00 00 00 00 00 90
1C60: 00 00 00 00 00 00 00 00 98
1C68: 00 00 00 00 00 00 00 00 A0
1C70: 00 00 00 00 8E 02 1D 0A 69
1C78: 8D FB 1C A9 80 8D 93 19 0F
1C80: 20 7C 1A 8D D7 1C 8E C5 16
1C88: 1C 4E 93 19 A5 E2 48 8D 3D
1C90: AC 1C A0 55 A2 54 4E D7 6A
1C98: 1C 90 02 88 E8 8E ED 1C 45
1CA0: 8C D9 1C A2 00 8E E4 1C 63
1CA8: 6E E4 1C A0 00 C0 C0 F0 54
1CB0: 4D 8D 80 C0 B9 00 D8 85 14
1CB8: 26 B9 00 D9 0D 97 19 85 8E
1CC0: 27 8D 88 C0 A0 00 F0 0E 02
1CC8: 1E 34 1C 10 02 38 90 18 CB
1CD0: 3E 35 1C 88 D0 F2 A0 00 15
1CD8: 8D 54 C0 B1 26 5D 34 1C 4B
1CE0: 91 26 E8 A9 00 10 05 C8 36
1CE8: C0 28 F0 0A 8D 54 C0 B1 3B
1CF0: 26 5D 34 1C 91 26 E8 EE C1
1CF8: AC 1C E0 10 90 AD A5 E0 13
1D00: 18 69 0E 85 E0 90 02 E6 EE
1D08: E1 68 85 E2 8D 54 C0 60 CB
1D10: 20 F8 E6 86 60 A9 2C 20 FF
1D18: C0 DE 20 F8 E6 86 61 A9 BB
1D20: 2C 20 C0 DE 20 67 DD 20 F8
1D28: 52 E7 A5 51 85 63 A5 50 A4
1D30: 85 62 20 B7 00 C9 C5 D0 C8
1D38: 0C 20 C0 DE 20 A1 19 86 C6
1D40: E2 84 E0 85 E1 A5 61 0A F3
1D48: A8 88 30 08 B1 62 99 34 FD
1D50: 1C 4C 49 1D A5 61 A6 60 07
1D58: 4C 74 1C 00 00 00 00 00 59
1D60: 00 00 A5 00 85 18 A5 01 28
1D68: 85 19 B9 00 00 85 15 B9 DC
1D70: 01 00 85 16 A9 00 85 17 AC
1D78: 95 00 95 01 95 02 A0 08 3E
1D80: 46 19 66 18 90 13 18 A5 19
1D88: 15 75 00 95 00 A5 16 75 3C
1D90: 01 95 01 A5 17 75 02 95 53
1D98: 02 06 15 26 16 26 17 88 5A
1DA0: D0 DE 60 8D 61 1D 78 8D DD
1DA8: 09 C0 AD 60 1D 85 1B A9 32
1DB0: 00 8D 5D 1D 85 08 85 05 28
1DB8: 85 02 85 0F 85 10 85 11 60
1DC0: AE 5E 1D 86 00 8E 5B 1D 04
1DC8: AD 5F 1D 85 01 8D 5C 1D C1
1DD0: 09 08 8D 7F 1D C9 08 D0 2C
1DD8: 0D BD 00 15 85 03 BD 00 0E
1DE0: 14 85 04 4C ED 1D A0 00 F0
1DE8: A2 03 20 62 1D A6 1B BD D6
1DF0: 00 15 85 06 BD 00 14 85 1D
1DF8: 07 A0 06 A2 09 20 62 1D 74
1E00: 18 A5 03 65 06 85 0C A5 6C
1E08: 04 65 07 85 0D A5 05 69 4B
1E10: 00 4A 85 0E 66 0D 66 0C B0
1E18: 46 0E 66 0D 66 0C 38 A5 12
1E20: 0C E5 09 85 0C A5 0D E5 4C
1E28: 0A 85 0D A5 0E E5 0B 85 6A
1E30: 0E 06 03 26 04 26 05 06 80
1E38: 06 26 07 26 08 06 09 26 D4
1E40: 0A 26 0B A5 00 18 65 1B 0D
1E48: 85 12 A5 01 69 00 85 13 F9
1E50: 8D 08 C0 20 D0 1E 8D 09 92
1E58: C0 A5 0E 10 2C EE 5D 1D 16
1E60: 18 A5 0F 65 03 85 0F A5 3C
1E68: 10 65 04 85 10 A5 11 65 7D
1E70: 05 85 11 18 A5 0C 65 0F 6B
1E78: 85 0C A5 0D 65 10 85 0D 83
1E80: A5 0E 65 11 85 0E 4C BA 88
1E88: 1E AD 5B 1D D0 03 CE 5C 09
1E90: 1D CE 5B 1D 38 A5 09 E5 9C
1E98: 06 85 09 A5 0A E5 07 85 30
1EA0: 0A A5 0B E5 08 85 0B 38 AF
1EA8: A5 0C E5 09 85 0C A5 0D BC
1EB0: E5 0A 85 0D A5 0E E5 0B 20
1EB8: 85 0E A5 12 38 E9 01 85 02
1EC0: 12 A5 13 E9 00 85 13 90 3D
1EC8: 03 4C 50 1E 8D 08 C0 60 F3
1ED0: A5 E0 85 E6 18 6D 5B 1D 81
1ED8: 85 E0 A5 E1 85 E7 6D 5C E5
1EE0: 1D 85 E1 A5 E2 85 E3 18 B0
1EE8: 6D 5D 1D 85 E2 48 A5 E3 96
1EF0: 38 ED 5D 1D 48 AD 61 1D 1B
1EF8: 29 01 F0 03 20 6C 1A 68 A7
1F00: 85 E2 AD 61 1D 29 04 F0 0C
1F08: 03 20 6C 1A A5 E6 38 ED 26
1F10: 5B 1D 85 E0 A5 E7 ED 5C 07
1F18: 1D 85 E1 AD 61 1D 29 08 37
1F20: F0 03 20 6C 1A 68 85 E2 C2
1F28: AD 61 1D 29 02 F0 03 20 C5
1F30: 6C 1A A5 E3 85 E2 A5 E7 09
1F38: 85 E1 A5 E6 85 E0 60 20 65
1F40: 67 DD 20 52 E7 A5 51 8D D8
1F48: 5F 1D A5 50 8D 5E 1D A9 01
1F50: 2C 20 C0 DE 20 F8 E6 8E F3
1F58: 60 1D A9 2C 20 C0 DE 20 E7
1F60: F8 E6 8A 48 20 B7 00 C9 54
1F68: C5 D0 0C 20 C0 DE 20 A1 A4
1F70: 19 86 E2 84 E0 85 E1 2C 8E
1F78: 98 19 10 0F A0 02 4E 5F 45
1F80: 1D 6E 5E 1D 46 E1 66 E0 ED
1F88: 88 D0 F3 68 20 A3 1D 2C 3A
1F90: 98 19 10 08 06 E0 26 E1 C5
1F98: 06 E0 26 E1 8D 54 C0 60 94
1FA0: A0 D3 D4 00 00 00 00 00 BE
```

Program 2: Chrome Demonstration

For instructions on entering this program, please refer to "COMPUTE!'s Guide to Typing In Programs" elsewhere in this issue.

```
18 10 HGR 3
71 20 DOUBLE
CD 30 HPLOT 0,0 TO 559,159
9A 40 HCOLOR= 3: HPLOT 0,0: HCOL
      OR= 0
8C 50 A = 768
66 60 FOR S = 1 TO 20
91 70 SHAPE 8,8,A
AB 80 NEXT
93 100 DATA 127,0,65,0,65,0,65,0
      ,65,0,65,0,127,0,0
8F 105 DATA 0
7C 200 REM  DRAW CIRCLES
E2 210 HPLOT 280,96
0D 220 HCOLOR= 3: HPLOT 280,96:
      HCOLOR= 0: REM SET MIDDLE
BE 225 REM /ELLIPSE/ WITHOUT "AT
       X,Y" IS FASTER
C7 230 FOR X = 1 TO 90 STEP 8
D0 240 ELLIPSE X * 2,X,15
06 260 NEXT
A2 300 HOME : VTAB 21
1C 310 PRINT "THIS IS A SIMPLE D
      EMO OF THE NEW GRAPHICS C
      OMMANDS
B2 320 PRINT TAB( 10)"AT THE UPP
      ER LEFTHAND CORNER OF THE
       SCREEN IS THE OUTPUT OF
      /SHAPE/
14 330 PRINT TAB( 10)"THE CIRCLE
      S ARE ACTUALLY ELLIPSES,
      PRODUCED BY /ELLIPSE/
F8 340 PRINT TAB( 10)"AND THE LI
      NE IS DRAW BY HPLOT... HI
      T <RETURN>";
D9 350 GET A$
AE 360 HOME : VTAB 21
EF 370 HGR 3
6B 380 PRINT "IN /COLOR/, THINGS
       LOOK LIKE THIS:
FB 390 COLOR
61 400 FOR X = 0 TO 15: COLOR= X
40 410 HPLOT 0,0 TO 30 * X,150
FF 420 NEXT
FC 430 COLOR= 12: REM BLUE
0F 440 ELLIPSE 250,70,15 AT 280,
      80
59 450 PRINT TAB( 8)"NOTE, THOUG
      H, THAT THE /SHAPE/ COMMA
      ND WORKS ON THE BIT LEVEL
      .
9A 460 PRINT TAB( 8)"/SHAPE/ AFF
      ECTS TWO ADJACENT BYTES O
      F VIDEO RAM PER 14-BIT LI
      NE"
E4 465 PRINT TAB( 8)"AND WAS ORI
      GINALLY INTENDED FOR MONO
      CHROME DRAWING ONLY";
72 470 PRINT "    HIT <RETURN>";:
       GET A$
67 480 TEXT : HOME
```

©

COMPUTE!
TOLL FREE
Subscription
Order Line
1-800-727-6937

MLX Machine Language Entry Program For Apple

Tim Victor, Editorial Programmer

To make it easier to enter machine language programs into your computer without typos, COMPUTE! is introducing its "MLX" entry program for the Apple II series. It's our best MLX yet. It runs on the II, II+, IIe, and IIc, and with either DOS 3.3 or ProDOS.

A machine language (ML) program is usually listed as a long series of numbers. It's hard to keep your place and even harder to avoid making mistakes as you type in the listing, since an incorrect line looks almost identical to a correct one. To make error-free entry easier, COMPUTE! generally lists ML programs for Commodore and Atari computers in a format designed to be typed in with a utility called "MLX." The MLX program uses a checksum system to catch typing errors almost as soon as they happen.

Apple MLX checks your typing on a line-by-line basis. It won't let you enter invalid characters or let you continue if there's a mistake in a line. It won't even let you enter a line or digit out of sequence. Best of all, you don't have to know anything about machine language to enter ML programs with MLX. Apple MLX makes typing ML programs almost foolproof.

Using Apple MLX

Type in and save some copies of Apple MLX on disk (you'll want to use MLX to enter future ML programs in COMPUTE!). It doesn't matter whether you type it in on a disk formatted for DOS 3.3 or ProDOS. Programs entered with Apple MLX, however, must be saved to a disk formatted with the same operating system as Apple MLX itself.

If you have an Apple IIe or IIc, make sure that the key marked CAPS LOCK is in the down position. Type RUN. You'll be asked for the starting and ending addresses of the ML program. These values vary for each program, so they're given at the beginning of the ML program listing and in the program's accompanying article. Find them and type them in.

The next thing you'll see is a menu asking you to select a function. The first is (E)NTER DATA. If you're just starting to type in a program, pick this. Press the E key, and the program asks for the address where you want to begin entering data. Type the first number in the first line of the program listing if you're just starting, or the line number where you left off if you've already typed in part of a program. Hit the RETURN key and begin entering the data.

Once you're in Enter mode, Apple MLX prints the address for each program line for you. You then type in all nine numbers on that line, beginning with the first two-digit number after the colon (:). Each line represents eight bytes and a checksum. When you enter a line and hit RETURN, Apple MLX recalculates the checksum from the eight bytes and the address. If you enter more or less than nine numbers, or the checksum doesn't exactly match, Apple MLX erases the line you just entered and prompts you again for the same line.

Invalid Characters Banned

Apple MLX is fairly flexible about how you type in the numbers. You can put extra spaces between numbers or leave the spaces out entirely, compressing a line into 18 keypresses. Be careful not to put a space between two digits in the middle of a number. Apple MLX will read two single-digit numbers instead of one two-digit number (F 6 means F and 6, not F6).

You can't enter an invalid character with Apple MLX. Only the numerals 0–9 and the letters A–F can be typed in. If you press any other key (with some exceptions noted below), nothing happens. This safeguards against entering extraneous characters. Even better, Apple MLX checks for transposed characters. If you're supposed to type in A0 and instead enter 0A, Apple MLX will catch your mistake.

Apple MLX also checks to make sure you're typing in the right line. The address (the number to the left of the colon) is part of the checksum recalculation. If you accidentally skip a line and try to enter incorrect values, Apple MLX won't let you continue. Just make sure you enter the correct starting address; if you don't, you won't be able to enter any of the following lines. Apple MLX will stop you.

Editing Features

Apple MLX also includes some editing features. The left- and right-arrow keys allow you to back up and go forward on the line that you are entering, so you can retype data. Pressing the CONTROL (CTRL) and D keys at the same time (*delete*) removes the character under the cursor, shortening the line by one character. Pressing CTRL-I (*insert*) puts a space under the cursor and shifts the rest of the line to the right, making the line one character longer. If the cursor is at the right end of the line, neither CTRL-D nor CTRL-I has any effect.

When you've entered the entire listing (up to the ending address that you specified earlier), Apple MLX automatically leaves Enter mode and redisplays the functions menu. If you want to leave Enter mode before then, press the RETURN key when Apple MLX prompts you with a new line address. (For instance, you may want to leave Enter mode to enter a program listing in more than one sitting; see below.)

Display Data

The second menu choice, (D)ISPLAY DATA, examines memory and shows the contents in the same format as the program listing. You can use it to check your work or to see how far you've gotten. When you press D, Apple MLX asks you for a starting address. Type in the address of the first line you want to see and hit RETURN. Apple MLX displays program lines until you press any key or until it reaches the end of the program.

Save And Load

Two more menu selections let you save programs on disk and load them back into the computer. These are (S)AVE FILE and (L)OAD FILE. When you press S or L, Apple MLX asks you for the filename. The first time you save an ML program, the name you assign will be the program's filename on the disk. If you press L and specify a filename that doesn't exist on the disk, you'll see a disk error message.

If you're not sure why a disk error has occurred, check the drive. Make sure there's a formatted disk in the drive and that it was formatted by the same operating system you're using for Apple MLX (ProDOS or DOS 3.3). If you're trying to save a file and see an error message, the disk might be full. Either save the file on another disk or quit Apple MLX (by pressing the Q key), delete an old file or two, then run Apple MLX again. Your typing should still be safe in memory.

cursor, shortening the line by one character. Pressing CTRL-I (*insert*) puts a space under the cursor and shifts the rest of the line to the right, making the line one character longer. If the cursor is at the right end of the line, neither CTRL-D nor CTRL-I has any effect.

When you've entered the entire listing (up to the ending address that you specified earlier), Apple MLX automatically leaves Enter mode and redisplays the functions menu. If you want to leave Enter mode before then, press the RETURN key when Apple MLX prompts you with a new line address. (For instance, you may want to leave Enter mode to enter a program listing in more than one sitting; see below.)

Display Data

The second menu choice, (D)ISPLAY DATA, examines memory and shows the contents in the same format as the program listing. You can use it to check your work or to see how far you've gotten. When you press D, Apple MLX asks you for a starting address. Type in the address of the first line you want to see and hit RETURN. Apple MLX displays program lines until you press any key or until it reaches the end of the program.

Save And Load

Two more menu selections let you save programs on disk and load them back into the computer. These are (S)AVE FILE and (L)OAD FILE. When you press S or L, Apple MLX asks you for the filename. The first time you save an ML program, the name you assign will be the program's filename on the disk. If you press L and specify a filename that doesn't exist on the disk, you'll see a disk error message.

If you're not sure why a disk error has occurred, check the drive. Make sure there's a formatted disk in the drive and that it was formatted by the same operating system you're using for Apple MLX (ProDOS or DOS 3.3). If you're trying to save a file and see an error message, the disk might be full. Either save the file on another disk or quit Apple MLX (by pressing the Q key), delete an old file or two, then run Apple MLX again. Your typing should still be safe in memory.

Apple MLX: Machine Language Entry Program

For instructions on entering this program, please refer to "COMPUTE!'s Guide to Typing In Programs" elsewhere in this issue.

```
80 100 N = 9: HOME : NORMAL : PR
       INT "APPLE MLX": POKE 34,
       2: ONERR GOTO 610
CC 110 VTAB 1: HTAB 20: PRINT "S
       TART ADDRESS";: GOSUB 530
       : IF A = 0 THEN PRINT CHR
       $ (7): GOTO 110
8C 120 S = A
EJ 130 VTAB 2: HTAB 20: PRINT "E
       ND ADDRESS  ";: GOSUB 530
       : IF S > = A OR A = 0 THE
       N PRINT CHR$ (7): GOTO 13
       0
20 140 E = A
B5 150 PRINT : PRINT "CHOOSE:(E)
       NTER DATA";: HTAB 22: PRI
       NT "(D)ISPLAY DATA": HTAB
        8: PRINT "(L)OAD FILE  (
       S)AVE FILE  (Q)UIT": PRIN
       T
AE 160 GET A$: FOR I = 1 TO 5: I
       F A$ < > MID$ ("EDLSQ",I,
       1) THEN NEXT : GOTO 160
93 170 ON I GOTO 270,220,180,200
       : POKE 34,0: END
AF 180 INPUT "FILENAME: ";A$: IF
        A$ < > "" THEN PRINT CHR
       $ (4);"BLOAD";A$;",A";S
A1 190 GOTO 150
38 200 INPUT "FILENAME: ";A$: IF
        A$ < > "" THEN PRINT CHR
       $ (4);"BSAVE";A$;",A";S;"
       ,L";(E - S) + 1
92 210 GOTO 150
C2 220 GOSUB 590: IF B = 0 THEN
       150
9E 230 FOR B = B TO E STEP 8:L =
        4:A = B: GOSUB 580: PRIN
       T A$;": ";:L = 2
85 240 FOR F = 0 TO 7:V(F + 1) =
        PEEK (B + F): NEXT : GOS
       UB 560:V(9) = C
F2 250 FOR F = 1 TO N:A = V(F):
       GOSUB 580: PRINT A$" ";:
       NEXT : PRINT : IF PEEK (4
       9152) < 128 THEN NEXT
94 260 POKE 49168,0: GOTO 150
CC 270 GOSUB 590: IF B = 0 THEN
       150
48 280 FOR B = B TO E STEP 8
A6 290 HTAB 1:A = B:L = 4: GOSUB
        580: PRINT A$;": ";: CAL
       L 64668:A$ = "":P = 0: GO
       SUB 330: IF L = 0 THEN 15
       0
F9 300 GOSUB 470: IF F < > N THE
       N PRINT CHR$ (7);: GOTO 2
       90
27 310 IF N = 9 THEN GOSUB 560:
       IF C < > V(9) THEN PRINT
       CHR$ (7);: GOTO 290
72 320 FOR F = 1 TO 8: POKE B +
       F - 1,V(F): NEXT : PRINT
       : NEXT : GOTO 150
8E 330 IF LEN (A$) = 33 THEN A$
       = O$:P = O: PRINT CHR$ (7
       );
22 340 L = LEN (A$):O$ = A$:O =
       P:L$ = "": IF P > 0 THEN
       L$ = LEFT$ (A$,P)
E0 350 R$ = "": IF P < L - 1 THE
       N R$ = RIGHT$ (A$,L - P -
        1)
55 360 HTAB 7: PRINT L$;: FLASH
       : IF P < L THEN PRINT MID
       $ (A$,P + 1,1);: NORMAL :
        PRINT R$;
7B 370 PRINT " ";: NORMAL
E6 380 K = PEEK (49152): IF K <
       128 THEN 380
C1 390 POKE 49168,0:K = K - 128
5B 400 IF K = 13 THEN HTAB 7: PR
       INT A$;" ";: RETURN
8A 410 IF K = 32 OR K > 47 AND K
        < 58 OR K > 64 AND K < 7
       1 THEN A$ = L$ + CHR$ (K)
        + R$:P = P + 1
C1 420 IF K = 4 THEN A$ = L$ + R
       $
5F 430 IF K = 9 THEN A$ = L$ + "
        " + MID$ (A$,P + 1,1) +
       R$
0A 440 IF K = 8 THEN P = P - (P
       > 0)
93 450 IF K = 21 THEN P = P + (P
       < L)
9D 460 GOTO 330
37 470 F = 1:D = 0: FOR P = 1 TO
        LEN (A$):C$ = MID$ (A$,P
       ,1): IF F > N AND C$ < >
       " " THEN RETURN
B8 480 IF C$ < > " " THEN GOSUB
       520:V(F) = J + 16 * (D =
       1) * V(F):D = D + 1
5F 490 IF D > 0 AND C$ = " " OR
       D = 2 THEN D = 0:F = F +
       1
08 500 NEXT : IF D = 0 THEN F =
       F - 1
17 510 RETURN
B5 520 J = ASC (C$):J = J - 48 -
        7 * (J > 64): RETURN
AB 530 A = 0: INPUT A$:A$ = LEFT
       $ (A$,4): IF LEN (A$) = 0
        THEN RETURN
6F 540 FOR P = 1 TO LEN (A$):C$
       = MID$ (A$,P,1): IF C$ <
       "0" OR C$ > "9" AND C$ <
       "A" OR C$ > "Z" THEN A =
       0: RETURN
2D 550 GOSUB 520:A = A * 16 + J:
        NEXT : RETURN
28 560 C = INT (B / 256):C = B -
        254 * C - 255 * (C > 127
       ):C = C - 255 * (C > 255)
20 570 FOR F = 1 TO 8:C = C * 2
       - 255 * (C > 127) + V(F):
       C = C - 255 * (C > 255):
       NEXT : RETURN
DA 580 I = FRE (0):A$ = "": FOR
       I = 1 TO L:T = INT (A / 1
       6):A$ = MID$ ("0123456789
       ABCDEF",A - 16 * T + 1,1)
        + A$:A = T: NEXT : RETUR
       N
1F 590 PRINT "FROM ADDRESS ";: G
       OSUB 530: IF S > A OR E <
        A OR A = 0 THEN B = 0: R
       ETURN
0D 600 B = S + 8 * INT ((A - S)
       / 8): RETURN
B6 610 PRINT "DISK ERROR": GOTO
       150
```

©

All the programs in this issue are available on the ready-to-load *COMPUTE! Disk*.
To order a one-year (four-disk) subscription, call toll free
1-800-727-6937
Please specify which computer you are using.